

Title: Roots vs robots

Presented by:

Eeshah Rashid, Fatima Zaman, Hiba Abrar, Javeria Soomro

Course code: CT-175

Course title: Programming Fundamentals

Department: BCIT

Institution: NED University of Engineering and Technology

Submission date: 19th November' 2024

Contents

- **Introduction**
- **Background Study**
- **Technical tools**
- **Methodology**
- **Risk Factors**
- **Limitations**
- **Future scope**
- **Overall discussion**
- **Conclusion**
- **References**
- **Contributions**

Executive Summary

Roots vs robots is a defence game developed in C programming language and it is designed to engage players in a real-time battle between roots (plants) and robots. The game includes key elements such as grid-based gameplay, automated robot movements, and plant defences like Pea Shooters. Players strategically place plants on the grid to defend against incoming waves of robots. The project demonstrates programming practices, including event-driven updates and collision detection. The system serves as a foundation for further improvements like more diverse gameplay mechanics and advanced user interaction.

Introduction

This project revolves around a dynamic defence strategy where players place plants on a grid to defend against invading robots. The primary objective is to prevent the robots from reaching the leftmost edge of the grid, using plant-based weapons like Pea Shooters which shoot peas (bullets).

This game in C language, showcases essential programming concepts such as loops, structures, collision detection, and timers. The simplicity of the system emphasizes logic and functionality.

Background Study

Roots vs robots is heavily inspired by “There will come soft rains” by Ray Bradbury which is a short story set in a future where technological advancements caused the extinction of the human race and with time, nature claims back its land.

Roots Vs robots, like the story, includes themes of survival with the idea of nature (plants) prevailing over technology (robots).

The game involves strategic planning and quick decision-making.

The game's core mechanics are grid-based plant placement and enemy movement and they highlight efficient handling of game state through data structures and loops. It builds upon foundational programming practices while focusing on a simplified version of the gameplay to ensure smooth operation and logic clarity. This approach not only makes it suitable for beginners but also paves the way for future enhancements.

Significance of the project:

The significance of this gaming project is the fact that it showcases essential programming concepts such as event handling and collision detection in C language which makes it a valuable learning experience for new developers. Furthermore, the project serves as a foundation for future enhancements, fostering creativity and collaboration in game development.

Technical Tools

The tools used to develop this project are listed below:

1. Home Screen with “How to Play”

This is the **entry point** of the game where players see the game title, instructions, and a prompt to start. It transitions to the gameplay or difficulty selection screen.

Key Features:

- Displays the title and instructions.
- Waits for the player to press ENTER to proceed.

Explanation:

// Home Screen and How to Play Instructions

```

if (!gameRunning) {
    BeginDrawing();
    ClearBackground(SKYBLUE);
    DrawText("ROOTS VS ROBOTS", 200, 150, 40, DARKGRAY);
    DrawText("Press ENTER to Start", 250, 300, 20, DARKGRAY);
    DrawText("How to Play:", 50, 400, 20, DARKGRAY);
    DrawText("- Click to place plants at any position", 50, 430, 20, DARKGRAY);
    DrawText("- Prevent robots from reaching the left!", 50, 460, 20, DARKGRAY);

    if (IsKeyPressed(KEY_ENTER)) {
        DifficultyScreen(); // Transition to difficulty selection
    }
    EndDrawing();
}

```

2. Difficulty Selection (DifficultyScreen)

Displays a menu to let players choose between Easy and Hard modes, adjusting the time and robot speed accordingly.

Code:

```

void DifficultyScreen() {
    while (!gameRunning && !gameOver) {
        BeginDrawing();
        ClearBackground(BLUE);
        DrawText("Select Difficulty:", 250, 150, 40, DARKGRAY);
        DrawText("Press 1 for Easy", 250, 200, 20, DARKGRAY);
        DrawText("Press 2 for Hard", 250, 230, 20, DARKGRAY);

        if (IsKeyPressed(KEY_ONE)) {
            gameTime = EASY_TIME;
            robotSpeed = EASY_ROBOT_SPEED;
            gameRunning = true;
            InitGame();
        }
        if (IsKeyPressed(KEY_TWO)) {
            gameTime = HARD_TIME;
            robotSpeed = HARD_ROBOT_SPEED;
            gameRunning = true;
            InitGame();
        }

        EndDrawing();
    }
}

```

3. Plant Placement (Input)

Players click on a lane to place a plant, which will defend against incoming robots.

Key Features:

- Detects mouse input (IsMouseButtonPressed).
- Ensures only one plant can be placed per lane.

Explanation:

```
void PlacePlant(Vector2 position) {
    for (int i = 0; i < LANE_COUNT; i++) {
        if (!plants[i].active && position.y > i * 100 && position.y < (i + 1) * 100) {
            plants[i].position = (Vector2){50, i * 100 + 50}; // Place plant in the selected lane
            plants[i].active = true;
            break;
        }
    }
}
```

```
// Mouse Input for Plant Placement
if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
    PlacePlant(GetMousePosition());
}
```

4. Spawning Robots

Robots are periodically spawned in random lanes and move towards the player's base.

Key Features:

- Robots appear in random lanes.
- Their speed and health are based on the difficulty level.

Explanation:

```
void SpawnRobot() {
    for (int i = 0; i < MAX_ROBOTS; i++) {
        if (!robots[i].active) {
            robots[i].position = (Vector2){SCREEN_WIDTH, GetRandomValue(0, LANE_COUNT
- 1) * 100 + 50};
            robots[i].active = true;
            robots[i].health = 3; // Default health
            robots[i].speed = robotSpeed; // Speed based on difficulty
            break;
        }
    }
}
```

```
// Periodic Robot Spawning
```

```

if (remainingTime > 0 && GetRandomValue(0, 100) < 10) { // 10% chance per update
    SpawnRobot();
}

```

5. Collision Detection (Projectiles and Robots)

Projectiles shot by plants collide with robots, reducing their health. If a robot reaches the left side, the base's health is reduced.

Key Features:

- Detects collision between projectiles and robots.
- Checks if a robot reaches the base.

Explanation:

```

void CheckCollisions()
{
    for (int i = 0; i < MAX_PROJECTILES; i++)
    {
        if (projectiles[i].active)
        {
            for (int j = 0; j < MAX_ROBOTS; j++)
            {
                if (robots[j].active && CheckCollisionCircleRec(projectiles[i].position, 5,
(Rectangle){robots[j].position.x, robots[j].position.y, 40, 40}))
                {
                    robots[j].health--; // Decrease robot health on collision
                    projectiles[i].active = false; // Deactivate the projectile
                    if (robots[j].health <= 0)
                    {
                        robots[j].active = false;
                        score += 10; // Increase score when robot is defeated
                    }
                    break;
                }
            }
        }
    }
}

void EndGame(const char *message)
{
    gameOver = true;
    strcpy(endMessage, message);
}

```

6. Timer and Remaining Time

The timer decreases as the game progresses. The player must survive until the timer reaches zero.

Key Features:

- Updates time each second.
- Ends the game when the timer hits zero.

Explanation:

```
// Timer Update
timeSinceLastUpdate += GetFrameTime();
if (timeSinceLastUpdate >= secondsPerUpdate) {
    remainingTime -= 1;
    timeSinceLastUpdate = 0.0f;

    if (remainingTime <= 0) {
        EndGame("Congratulations: You Survived!");
    }
}
```

7. Game Over and End Screen

If the base health reaches zero or the player survives the timer, the game ends and displays a victory or defeat message.

Key Features:

- Displays a message based on the game's outcome.
- Allows restarting the game with ENTER.

Explanation:

```
void EndGame(const char *message) {
    gameOver = true;
    strcpy(endMessage, message);
}

// Game Over Screen
if (gameOver) {
    BeginDrawing();
    ClearBackground(SKYBLUE);
    DrawText(endMessage, 250, 200, 40, DARKGRAY);
    DrawText("Press ENTER to Restart", 250, 300, 20, DARKGRAY);

    if (IsKeyPressed(KEY_ENTER)) {
        gameOver = false;
        InitGame();
    }
}
```

```
}  
  
    EndDrawing();  
}
```

Programming Language:

C language:

C is a general-purpose programming language. By design, C's features cleanly reflect the capabilities of the targeted CPUs. C is commonly used on computer architectures that range from the largest supercomputers to the smallest microcontrollers and embedded systems.

Development environment:

Integrated development environment (IDEs).

IDE's such as visual studio code for writing and debugging the C code.

Libraries:

Stdio.h

Stdlib.h

Stdbool.h

Unistd.h

Raylib.h

Methodology

The development of the gaming project follows a systematic approach based on an efficient and well-structured process. The methodology consists of the following phases:

Requirement Analysis

Objective: To identify the functional and technical requirements for the game.

Activities:

Determining and defining the game mechanics. The game mechanics include player objectives, enemy behaviour and interactions.

Outcome: A detailed document outlining gameplay mechanics, assets, and performance criteria.

System Design

Objective: To design the game's structure and logic based on the requirements.

Activities:

- **System architecture design:** Create the game's architecture, including entity interactions (e.g., plants, robots, projectiles).
- **Development:** Develop algorithms for movement, collision detection, and event handling.
- **User interface design:** Design the user interface for clarity and ease of navigation.

Outcome: A system design document with flowcharts and logic diagrams detailing game mechanics and structure.

Development

Objective: To implement the game using the C programming language.

Activities:

- **Environment setup:** The development environment was set up with necessary compilers and libraries.
- **Coding:** Code game functionalities, including object initialization, updates, and rendering.
- **Module integration:** Integrate each component and ensure everything works smoothly.

Outcome: A functional prototype of the game.

Testing

Objective: To ensure the game works correctly and meets the requirements.

Activities:

- **Unit testing:** Testing different parts of the code to check functionality.
- **Integration testing:** Testing the integration of different components of the code to ensure the work together seamlessly.

Outcome: A gaming project that is fully tested.

Maintenance and Updates

Objective: To keep the game engaging and functional over time.

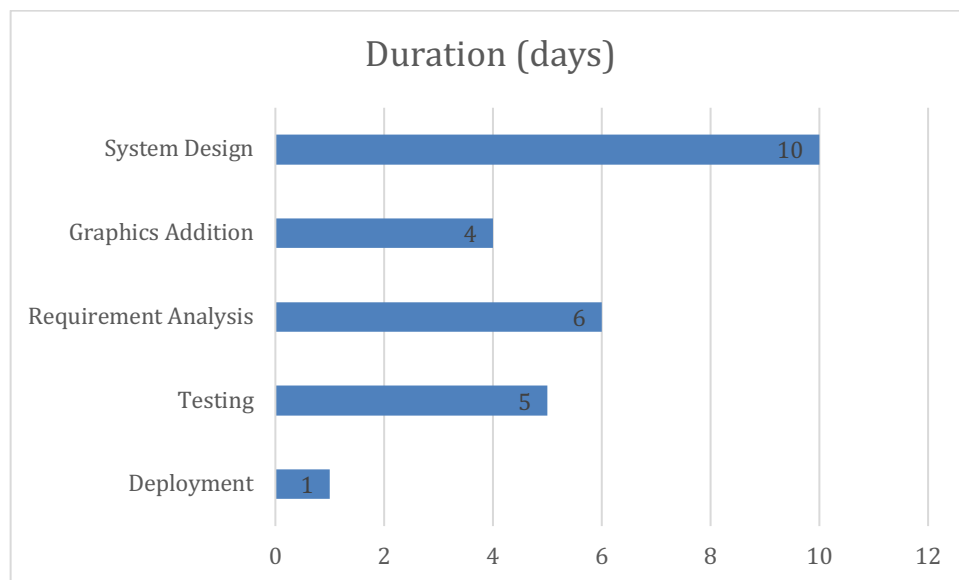
Activities:

- **Bug fixes:** Fix bugs and optimise performance based on player feedback.
- **Future enhancements:** Add new features, such as additional enemy types or levels.
- **Regular updates:** Provide regular updates to maintain compatibility with evolving systems.

Outcome: A continuously improved game that remains enjoyable for players.

Time and duration

The project phases and their respective phases and listed down below:

**Risk Factors**

The risk factors include:

1) Code Complexity

The code has a lot of repetitive parts and global variables. This can make it harder to add new features without introducing bugs.

2) Performance Issues

As more entities like robots, plants, and bullets are added, the game might slow down since it checks everything one by one.

3) Platform Dependency

Commands like `system("cls")` and `sleep ()` might not work correctly on all systems, especially on Windows without adjustments.

Limitations

There are a few limitations of this game and they are:

1) Few Games Features

The game currently has only two plant types and 2 robot types.

There are no special mechanics like resource collection or different attack patterns.

2) Hardcoded Settings

The grid size, delays, and number of plants or robots are fixed in the code so if there is a need to customise the game, we would have to change the code.

3) No Save or Resume Option

Players cannot save the game or continue where they left off.

4) Collision Handling Issues

There might be errors or unexpected behaviour when multiple objects (plants, bullets, or enemies) are in the same spot.

Future scope

Roots vs robots have huge future potential and a few things that will improve it drastically are:

1) More Plant and Enemy Types

Addition of plants and robots with unique abilities (such as plants with new attacking strategies and robots with different abilities).

2) Sound Effects and Music

Addition of background music and sounds for actions like shooting, damage, and victory to make the game more engaging.

3) Save and Load Functionality

Allowing players to save their progress and resume the game later.

4) Scoring System

Introduction of a scoring system based on the number of enemies defeated and display a leaderboard for high scores.

5) Customization Options

Allowing players to select grid size, game speed, or the number of robots at the start without modifying the code.

6) Seasonal Themes and Challenges

Addition of themed variations (e.g., winter or space settings) and special challenges to keep the game fresh.

- 7) **Non-blocking input:** The current implementation uses a blocking input mechanism, which temporarily halts the game while waiting for user input. Introducing a non-blocking input system in future iterations could enable smoother gameplay by allowing real-time interactions without interrupting the game's flow. This enhancement presents a promising area for further development.

Discussion

In this project, a dynamic and interactive game was developed using the C programming language. The objective of this game was to provide a simple yet engaging experience, incorporating real-time updates, collision detection, and user interaction to simulate a battle between plants and enemy robots. The project aimed to enhance programming skills through the implementation of core concepts such as structures, arrays, loops, and event-based logic.

The key features implemented in the game include:

- 8) **Grid-based Gameplay:** The game operates on a 6x12 grid where players can strategically place plant units (Pea Shooters and Cacti) to counter enemy robots. The grid system allows for real-time updates and dynamic positioning of characters.

9) Character Roles and Abilities:

Plants:

Pea Shooters attack using bullets and deal damage to enemy robots.

Enemies:

Robots move across the grid toward the player's plants.

- 10) **Event-based Timers:** Each character's actions (movement, attacks) are controlled by timers to ensure balanced gameplay. Delays and triggers for movement, attacks, and collisions create an engaging flow.

- 11) **Collision Detection:** Comprehensive collision logic ensures interaction between units, such as:

- Bullets hitting robots and reducing their health.

- Thorns targeting shielded robots.
- Enemy units damaging plants upon contact.

12) Real-time Input and Updates: The game allows players to add plants during gameplay, start or stop the game, and view the evolving state of the grid, offering a seamless user experience.

13) Error Handling and Constraints: Input validation ensures players cannot exceed the maximum number of plants or place units outside the grid boundaries.

The project serves as a practical implementation of core programming concepts, demonstrating an effective combination of structures, arrays, loops, and conditionals to build an interactive game. It also highlights the importance of modularity and event-driven programming for managing complex behaviours.

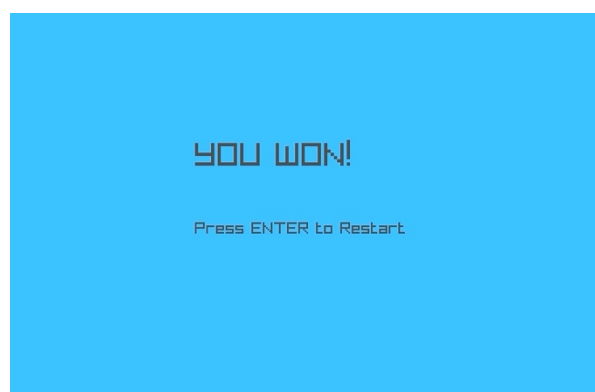
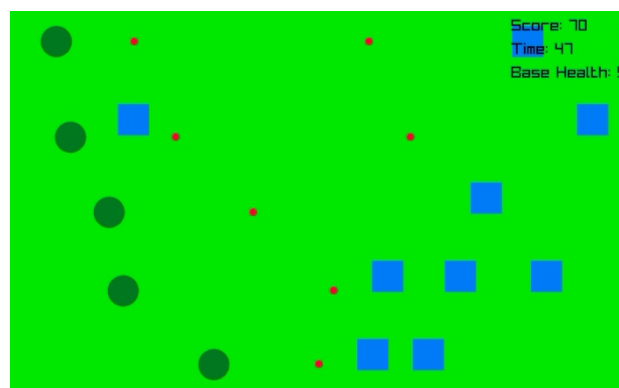
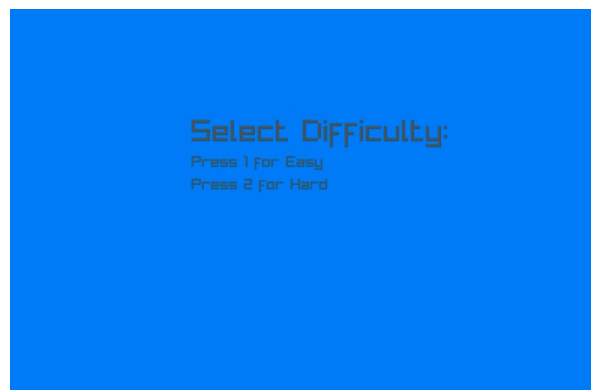
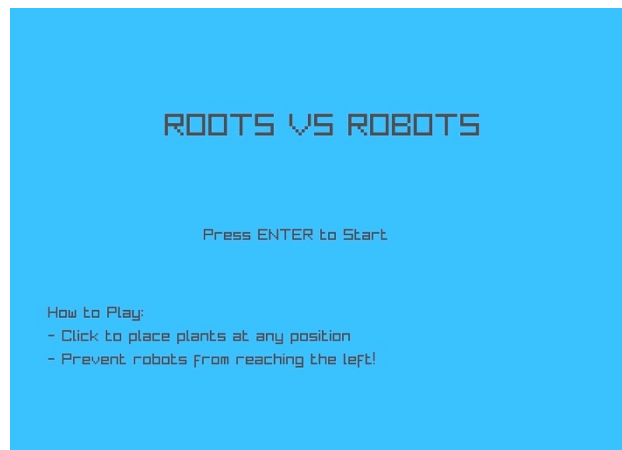
Conclusion

The development of this game successfully achieves its goal of creating an interactive and strategic experience using the C programming language. By incorporating features like real-time gameplay, collision detection, and structured event handling, the project shows the potential of C for game development and dynamic problem-solving.

This project not only serves as an engaging game but also provides a solid foundation for future improvements.

Overall, this project demonstrates the effective application of programming principles to build a functional and entertaining game. It provides a stepping stone for further exploration into advanced game development techniques which emphasises the value of creativity and problem-solving in programming.

Output



References

- <https://www.geeksforgeeks.org/>
- <https://stackoverflow.com/>
- <https://www.raylib.com/cheatsheet/cheatsheet.html>

Contributions

Eeshah Rashid - coding

Hiba Abrar - coding

Fatima Zaman – graphics addition

Javeria Soomro – graphics addition